



**הפקולטה להנדסת חשמל
ע"ע אנדרו וארנה ויטרבי**



**הטכניון
מכון טכנולוגי לישראל**

**מעבדה בהנדסת חשמל
044157 א'1**

ניסוי VGA - ממשק מסך וצלילים

חומר רקע

גרסה 1.31

ספטמבר 2022

תוכן עניינים

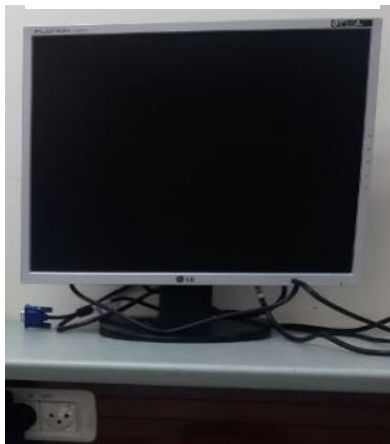
1.	הכרת הפלטפורמה VGA	3
1.1.	מהו VGA	3
1.2.	בקר ה-VGA – The VGA Controller	4
2.	יישום ה-VGA שבמעבדה	5
2.1.	כללי	5
2.2.	ייצוג צבעים ביישום	5
2.3.	מודול הרקע הסטטי back_ground_draw	6
2.4.	מודול התנועה של האובייקט smileyface_move	7
2.4.1.	מודול השליטה בתנועת האובייקט מהמקלדת	7
2.5.	המודולים לשרטוט האובייקט	8
2.5.1.	המודול לקביעת מלבן תוחם square_object	9
2.5.2.	המודול ליצירת ה-bit map - smileyBitMap	9
2.6.	בורר האובייקטים object_mux	10
3.	דוגמאות מימוש נוספות	11
3.1.	ציור קו	11
3.2.	חישוב המסלול לאובייקט בתנועה	11
3.3.	שכפול אובייקטים - פענוח XY	12
3.1.	אובייקטים נוספים מעל הרקע	13
3.1.1.	קביעת מיקום אקראי לאובייקט	13
3.2.	זיהוי התנגשות	14
3.2.1.	דוגמה של התנגשות כדור במסגרת	14
3.2.2.	טיפול בהתנגשות כפולה	15
3.3.	שכפול אובייקטים	15
3.4.	הוספת צלילים	17
3.4.1.	מחולל הצליל - גל סינוס	17
3.5.	תלת מימד – איזומטריה	19
4.	נספח - טבלת המרה של צבעים בסיסיים	20
	Hex to RGB color table	20
5.	נספח - טבלת המרה של כל צבעים	21

1. הכרת הפלטפורמה VGA

1.1 מהו VGA

VGA (Video Graphics Array) היא שיטה להעברת אותות וידאו אנלוגיים.

מחבר מסוג VGA הוא בעל 15 פינים מסודרים בשלוש שורות.



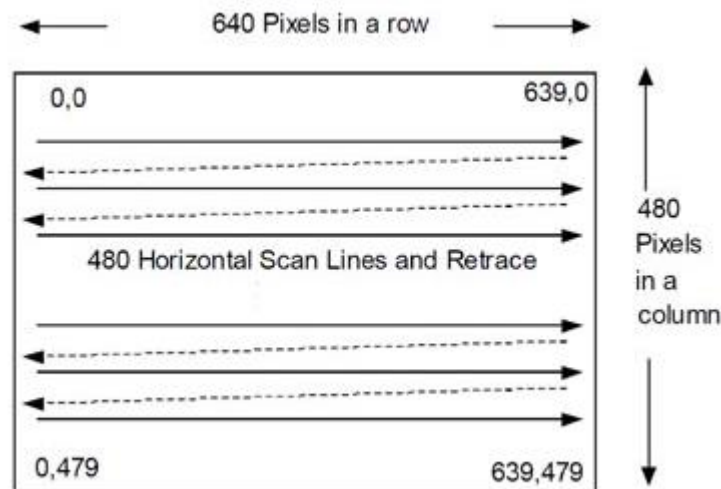
במעבדה מוצא ה-VGA של הכרטיס מתחבר אל המסך השלישי שנמצא בכל עמדת עבודה על המדף (מעל שני המסכים של העמדה).

במעבדה נעבוד עם רזולוציה של 640x480 פיקסלים (640 פיקסלים בשורה, ציר X ו-480 בעמודה, ציר Y). (משמעות המילה רזולוציה היא כושר הפרדה או כושר הבחנה).

- הפיקסל שמזוהה עם הקואורדינטה (0, 0) נמצא בפינה השמאלית העליונה של המסך.

- הפיקסל האחרון בעל הקואורדינטה (639, 479) נמצא בפינה הימנית התחתונה.

סריקת המסך מתבצעת משמאל לימין ומלמעלה למטה, כמודגם באיור להלן:



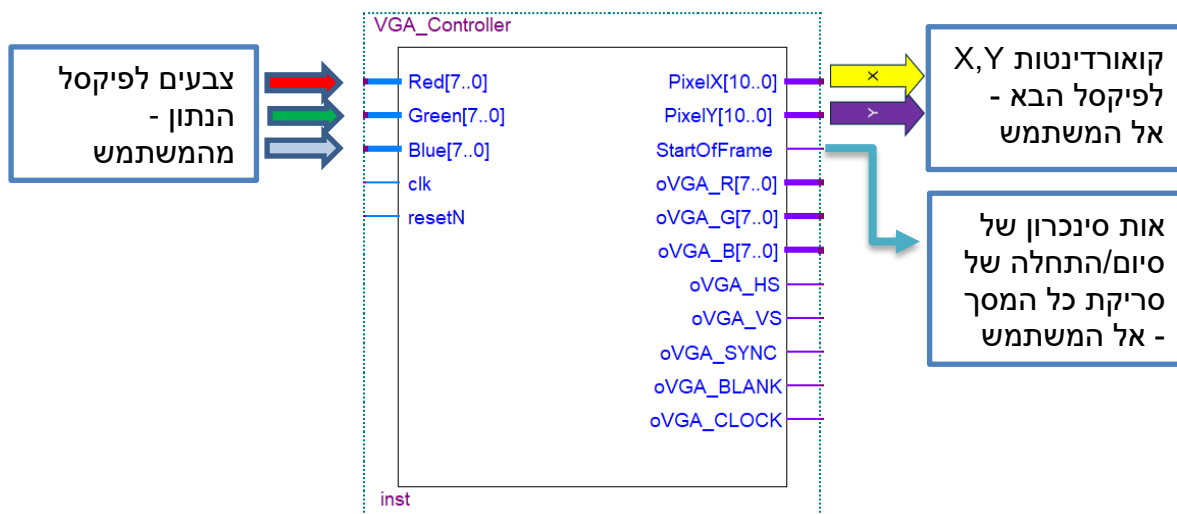
1.2 בקר ה- VGA – The VGA Controller

כרטיס העבודה שלנו כולל בקר VGA ומוצא וידאו שאפשר לחבר למסך VGA סטנדרטי.

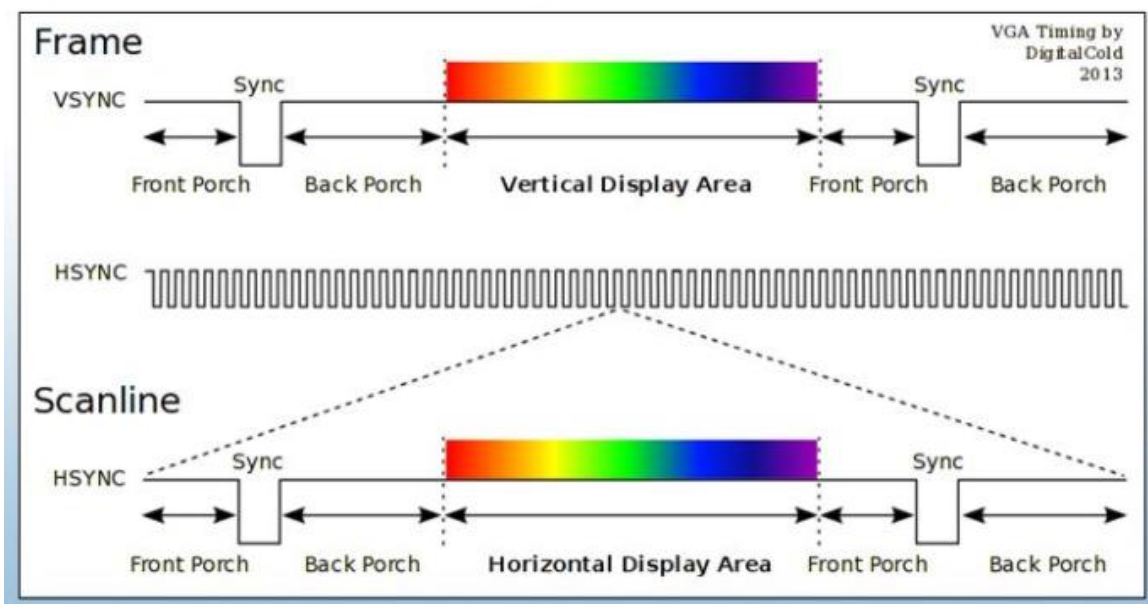
בקר ה- VGA תומך ברזולוציות מסך של 640x480.

בממשק עם המשתמש הבקר מקבל בכניסה (מהמשתמש) את קומבינציות הצבעים RGB שיש להקנות לפיקסל הנתון ומוציא (אל המשתמש) את:

- הקואורדינטות של הפיקסל הבא שיש להקנות לו צבע.
- האות startOfFrame – פולס בסיום הסריקה של כל המסך



הבקר מוציא גם את הצבע אל המסך (3 אותות של 8 ביט) ואותות תזמון לסינכרון אופקי H_SYNC, בסיום סריקת כל הפיקסלים בשורה, ואנכי V_SYNC, בסיום סריקת כל השורות. אותות אלה לא בשליטת המשתמש. התזמונים מודגמים באיור להלן.

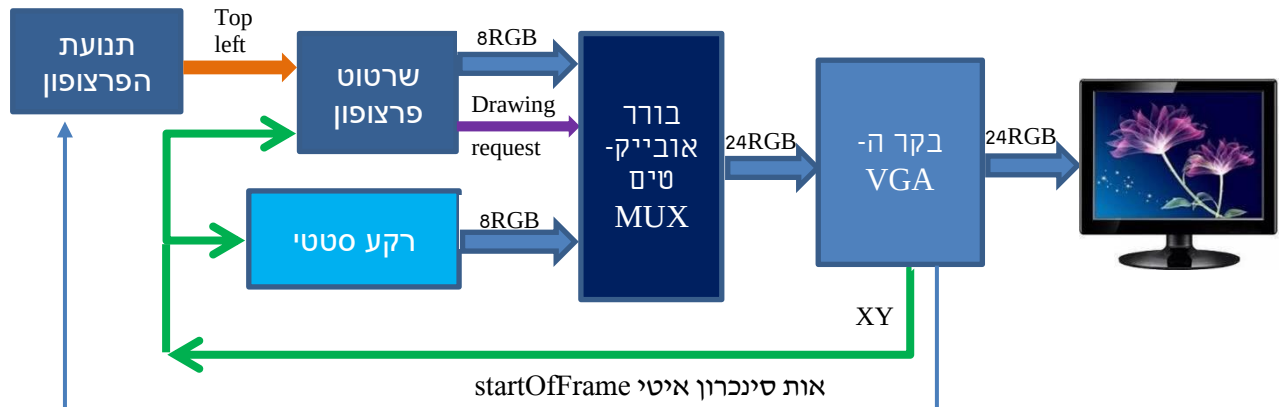


סרטון שמסביר איך זה עובד : <https://www.youtube.com/watch?v=3BJU2drrtCM&t=575s>

2. יישום ה- VGA שבמעבדה

2.1 כללי

נתון לכם יישום לדוגמה שמממש אובייקט בצורת פְּרָצוּפּוֹן (smiley) המבצע תנועה על רקע סטטי. יישום זה מורכב ממספר מודולים בסיסיים שמתוארים בצורה סכמתית להלן ויפורטו יותר בהמשך.

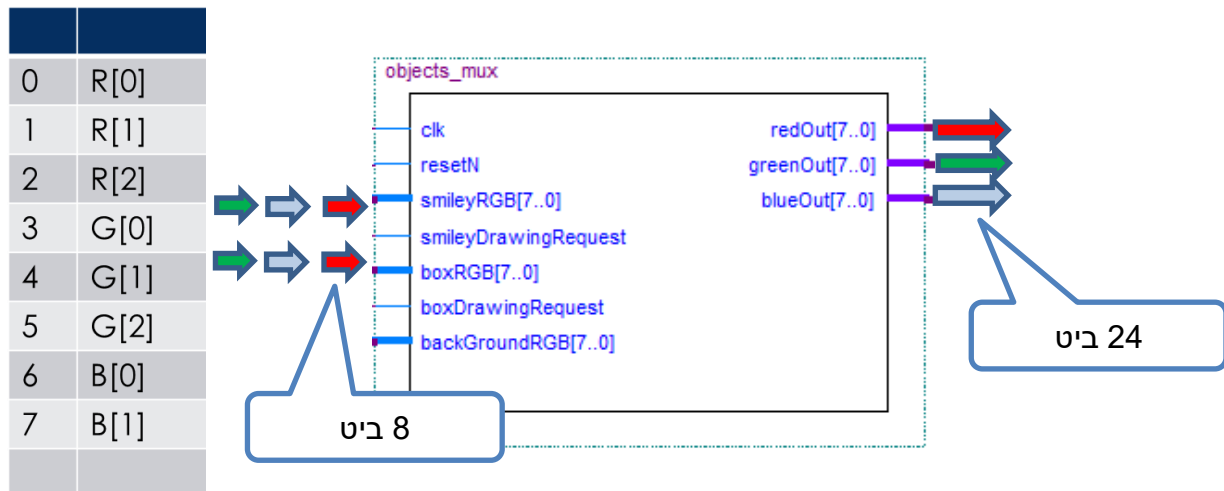


ביישום זה משתמשים בכניסות הרגילות:

- השעון הוא שעון הכרטיס, 50MHz, לכל היחידות.
- ה- resetN אסינכרוני, פעיל בנמוך וזהה לכל היחידות.

2.2 ייצוג צבעים ביישום

ביישום שלנו הצבעים מנוונים וצומצמו ל- 8 ביט בסך הכל, המאפשרים 3 ביט לאדום וירוק ו- 2 ביט לכחול. שיטת צמצום זו מאפשרת אפסון יעיל של BITMAP וחסכון בלוגיקה (חוסך גם בזמן קומפילציה חשוב מאד).



למשל, ייצוג של כמה צבעים סטנדרטים ביישום שלנו:

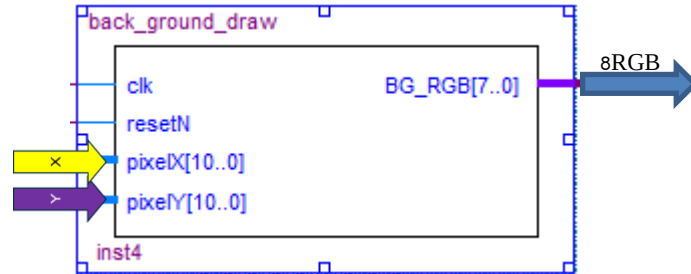
צבע	ייצוג סטנדרטי ב- 24 ביט ב- HEX	ייצוג ביישום ב- 8 ביט ב- HEX	ייצוג בינארי ביישום
צהוב	0xFFFF00	0xFC	11111100
CYAN	0x00FFFF	0x1F	00011111

ראה גם את הנספח בסוף המסמך.

2.3. מודול הרקע הסטטי back_ground_draw

מודול זה מממש רקע קבוע אך לא אחיד בנוי ממספר מלבנים בצבעים שונים.

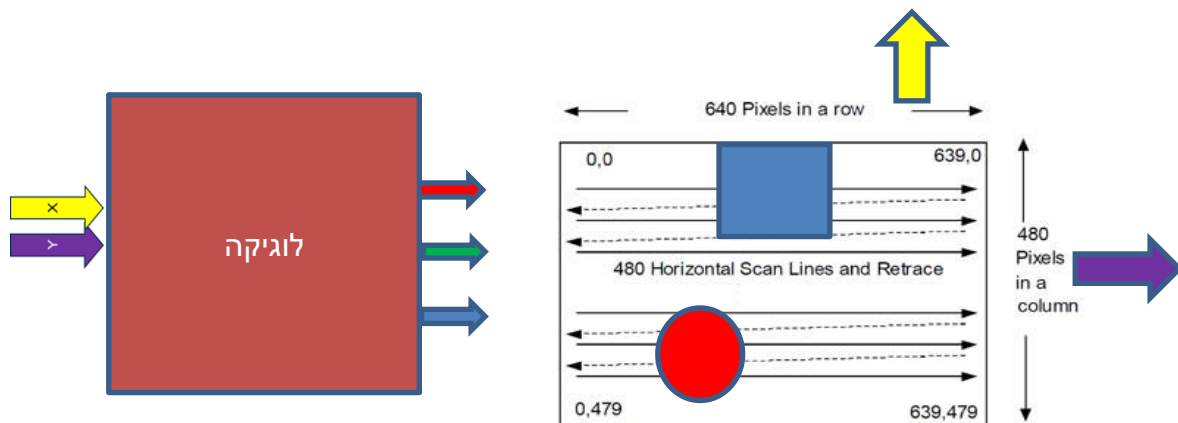
- מקבל את הקואורדינטה של הפיקסל הנתון
- מוציא את הצבע שיש להקנות לפיקסל (8 ביט)



מלבני הצבע מוגדרים בקוד באופן קשיח, ע"י גבולות מספריים קבועים בציר X ובציר Y, כאשר לכל תחום שכזה מוגדר צבע אחיד.

$$x_1 < X < x_2 \text{ \& } y_1 < Y < y_2$$

כמו כן, מצוירת גם מסגרת בצבע אחיד מסביב למסך.



המלצה: לא להשתמש ב- Bitmaps גדולים לרקע כי זה מאריך את זמני הקומפילציה לבלתי סבירים. ישנן עוד דרכים לייצר רקעים חסכוניים ויפים:

- יצירת רקע ממרצפות בגודל סביר שחוזרות על עצמם מספר פעמים (ראה פרק 3.3)
- חלוקת הרקע לאזורים בצבעים אחידים ושונים ובצורות גאומטריות שונות

שים לב: יש להגביל Bitmap לגודל 32*32 גם אם הוא קצת פחות יפה

2.4. מודול התנועה של האובייקט smileyface_move

מודול זה מחשב את תנועת האובייקט. כניסות המודול:

- startOfFrame - סינכרון איטי - אות מבקר ה-VGA של סריקת מסך שלם

- שתי כניסות נוספות שמגיעות מהמקלדת. ראו הסבר יותר מפורט בהמשך

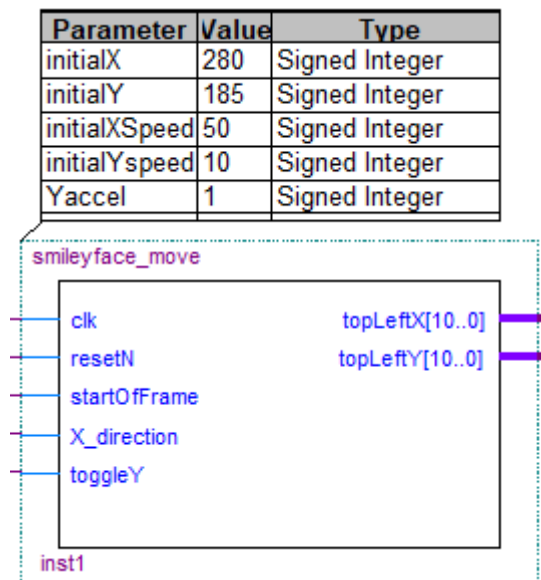
יציאות המודול הן:

- הקואורדינטות ההתחלתיות X,Y של האובייקט (פינה שמאלית עליונה שלו) במקום שחושב כעת

המודול מממש דוגמה של מסלול לפיו הקואורדינאה X משתנה לינארית מצד לצד של המסך והקואורדינאה Y משתנה למעלה ולמטה לפי גריטציה. כאשר הפרצופון מגיע לגבולות המסך הוא משנה כיוון כך שהוא אינו יוצא מן המסך.

האובייקט יתקדם פעם אחת בסריקת מסך מלאה, כדי למנוע טשטוש התמונה. לכן המודול יעדכן את מיקום האובייקט רק כאשר יתקבל אות startOfFrame מבקר ה-VGA, שמווה שעות איטי לצורך זה.

ניתן לקבוע את הפרמטרים של המודול חיצוניות.



פירוט נוסף על תנועת אובייקטים ניתן בפרק 3.2.

2.4.1. מודול השליטה בתנועת האובייקט מהמקלדת

מודול זה מאפשר שליטה בתנועת הפרצופון דרך הממשק למקלדת (שבנית במעבדת DEBUG).

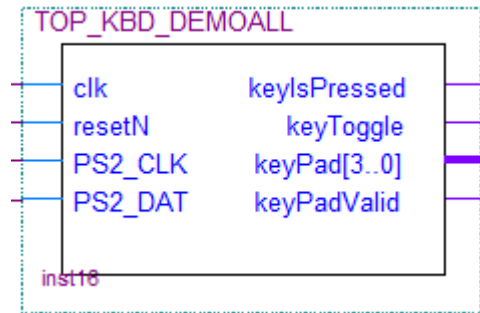
המודול מקבל כניסות מהמקלדת.

שתי היציאות, keyToggle ו-keyIsPressed, השולטות על תנועת הפרצופון, בדוגמה זו עובדות כ-

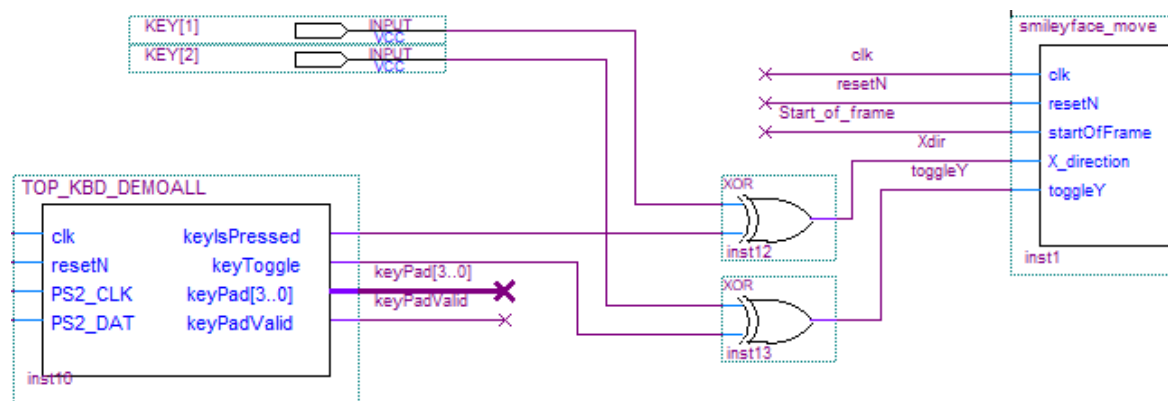
- toggle (מחליפה בין שני מצבים 0 ו-1 לוגי) ממופה למקש ENTER

- וכמקש פעיל (1 לוגי) כל עוד הוא לחוץ, ממופה למקש הרווח

היציאה keyPad מאפשרת להציג מספרים (0-9 ו-a-f) על המסך והיציאה keyPadValid נותנת אינדיקציה על מקש חוקי. בהמשך יוסבר על שימושים נוספים ביציאות אלה.

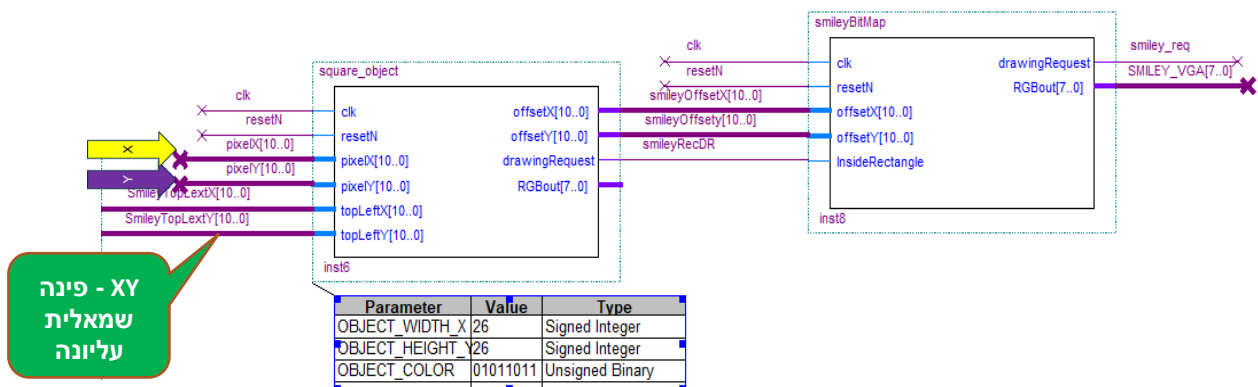


שיעור 2.5 שבמקביל לשליטה מהמקלדת מחוברים גם שני לחצנים של הכרטיס לכניסות של X_direction ו-Y_toggle, למקרה שאין בנמצא מקלדת.



2.5. המודולים לשרטוט האובייקט

כפי שהוסבר בפרק הקודם, האובייקט משורטט על המסך במקום שנקבע ע"י מודול התנועה. ישנם שני מודולים שאחראים לשרטוט האובייקט על המסך: square_object ו-smileyBitMap.



2.5.1. המודול לקביעת מלבן תוחם square_object

מודול זה בודק האם הקואורדינטה הנתונה נמצאת בתחום מלבני שתוחם את האובייקט בכללותו. אם כן, יש להציג אותה. ניתן לקבוע את מימדי המלבן וצבע הבסיס שלו ע"י פרמטרים חיצוניים. הכניסות של המודול הן:

- הקואורדינטה הנתונה (pixelX, pixelY)
- הקואורדינטה של מיקום הפינה השמאלית העליונה של האובייקט היציאות שלו הן:
- הקואורדינטה יחסית לפינה השמאלית העליונה בתוך מלבן הצורה
- בקשה להצגת הנקודה כי היא בתוך המלבן

2.5.2. המודול ליצירת ה-bit map - smileyBitMap

המודול משרטט את האובייקט במקום הנתון לפי bit map שמפרט את צורתו והצבעים שלו. הכניסות שלו:

- הקואורדינטה יחסית לפינה השמאלית העליונה בתוך מלבן הצורה
 - בקשה להצגת הנקודה כי היא בתוך המלבן
- היציאות שלו:

- הצבע של הפיקסל הנתון
- בקשת תצוגה (drawing request) אם צריך

בדוגמה הנתונה הפרצופון בגודל 26X26. ה-bit map נתון במערך דו-מימדי בשם **object_colors** בגודל הצורה, כאשר תא במערך מגדיר את הצבע לפיקסל הנתון. כמו שהוסבר קודם, ניתן להרכיב רק 256 צבעים, בפורמט RRRGGBB ביצוג הקסדצימאלי. קטע ממערך זה נתון להלן:

```
logic [0:OBJECT_HEIGHT_Y-1] [0:OBJECT_WIDTH_X-1] [7:0] object_colors = {
{8'hFF, 8'hFF, 8'hFF, 8'hFF, 8'hFF, 8'hFF, 8'hFF, 8'hFF, 8'hFF, 8'hFF, 8'hFF,
{8'hFF, 8'hFF, 8'hFF, 8'hFF, 8'hFF, 8'hFF, 8'hFF, 8'hFF, 8'hD5, 8'hD0, 8'hD0,
{8'hFF, 8'hFF, 8'hFF, 8'hFF, 8'hFF, 8'hFF, 8'hD5, 8'hD0, 8'hD4, 8'h64, 8'h8C,
{8'hFF, 8'hFF, 8'hFF, 8'hFF, 8'hFF, 8'hD0, 8'hD4, 8'hD9, 8'h8C, 8'hD9, 8'hFF,
{8'hFF, 8'hFF, 8'hFF, 8'hFF, 8'hD0, 8'hD4, 8'hD9, 8'hD9, 8'hFF, 8'hD9, 8'hD9,
{8'hFF, 8'hFF, 8'hFF, 8'hD0, 8'hD4, 8'hD9, 8'hD5, 8'hD4, 8'h40, 8'h04, 8'h9B, 8'h9B,
{8'hFF, 8'hFF, 8'hD0, 8'hD4, 8'hD9, 8'hD5, 8'hD4, 8'h40, 8'h29, 8'hFF, 8'hFF, 8'hFF,
{8'hFF, 8'hB6, 8'hD0, 8'hD9, 8'hD5, 8'hD9, 8'h40, 8'h29, 8'hFF, 8'hFF, 8'hFF,
{8'hFF, 8'hD0, 8'hD4, 8'hD9, 8'hB0, 8'h44, 8'h44, 8'hFF, 8'h96, 8'h0A, 8'h92,
```

צבע שקוף: ישנו פרמטר מקומי שמוסכם להוות צבע שקוף (אותו לא מציגים למרות שהוא בתחום המלבן). בדוגמה שלנו הצבע השקוף נבחר להיות הצבע הלבן (0xFF).

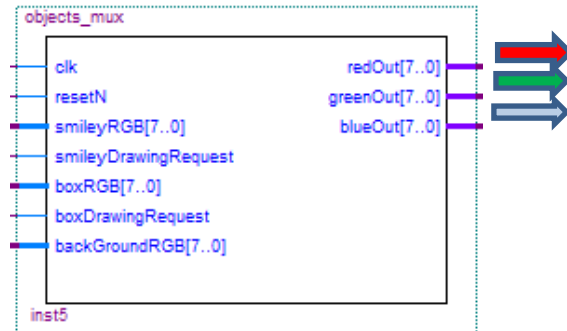
```
localparam logic [7:0] TRANSPARENT_ENCODING = 8'hFF ;
```

במקרה כזה אם באמת רוצים להציג צבע לבן בפיקסל מסויים, יש להציג צבע אחר, קרוב מאד ללבן, בדוגמה שלנו, 0xFE.

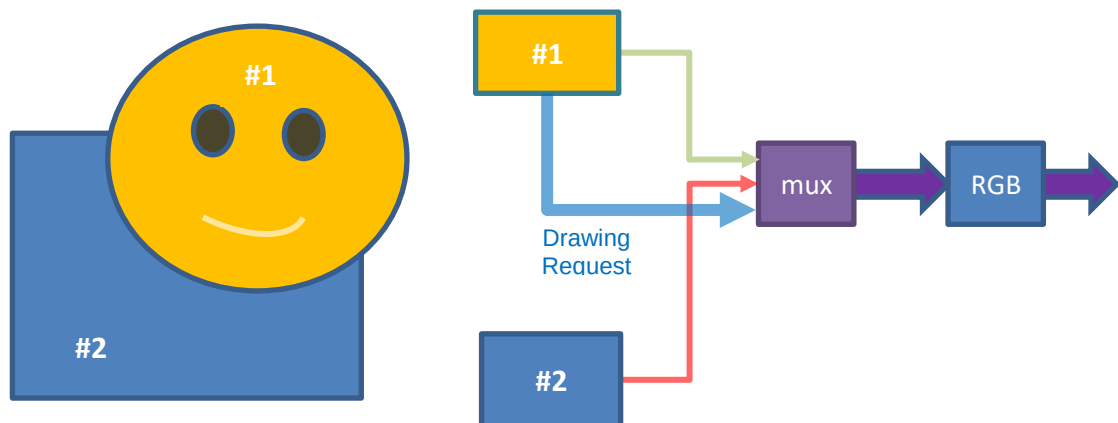
2.6. בורר האובייקטים *object_mux*

מודול זה קובע מה לצייר בפיקסל הנתון.

- מקבל את הצבעים ובקשות התצוגה (Drawing request) של הרקע, הפרצופון ואובייקטים נוספים שאולי יתווספו בהמשך
- מוציא את הצבעים של הפיקסל הנתון, בפורמט של 24 ביט (הרחבה ע"י הוספת אפסים מימין)



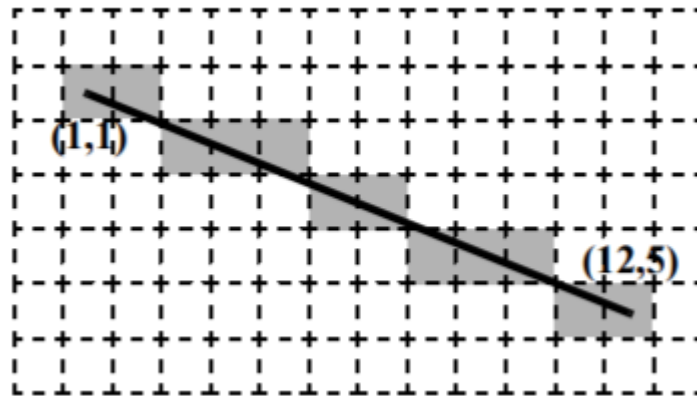
המודול קובע קדימות בעזרת כללי עדיפות פשוטים. הרקע מוצג תמיד (לכן אין צורך בבקשת תצוגה עבורו) והפרצופון יוצג מעל הרקע (הצבע שלו מוצג במקום צבע הרקע) כאשר המודול שלו מוציא בקשת תצוגה.



3. דוגמאות מימוש נוספות

3.1. ציור קו

ציור קו על מסך, פירושו לצבוע פיקסלים בין שתי נקודות, (x_1, y_1) ו- (x_2, y_2) . למשל, רוצים לצייר קו בין הנקודות $(1, 1)$ ו- $(12, 5)$. בדוגמה להלן מסומנים הפיקסלים שבקירוב נמצאים על הקו.



מיקום הפיקסלים מחושב בעזרת משוואת הקו (b – השיפוע הרצוי).

$$y = y_1 + b \cdot (x - x_1)$$

היות ועובדים עם מספרים שלמים, תוצאות החישובים מעוגלות למספר השלם הקרוב ביותר.

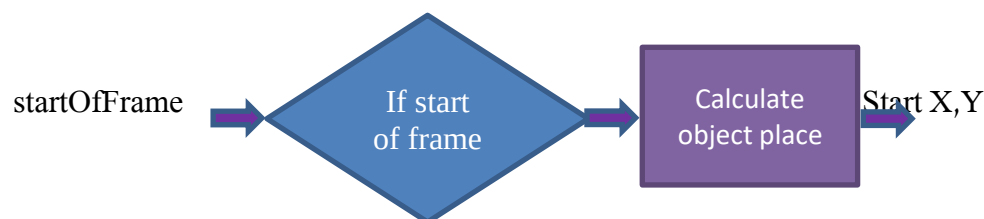
שימו לב שאם b הוא שבר, רצוי לבחור אותו כחזקה של 2, 2^{-n} , למשל $1/8$.

קו סטטי נוציא ע"י ציור כל נקודות הקו בכל סריקה של המסך.

3.2. חישוב המסלול לאובייקט בתנועה

תנועה של אובייקט נוצרת ע"י כך שכל פעם מציירים את האובייקט במיקום אחר, לאורך מסלול רצוי. סריקה שלמה של מסך אורכת $1/60$ שניה. זהו גם הזמן שלוקח להחליף את מיקום האובייקט המצויר ועדיין העין שלנו תראה תנועה רציפה.

השימוש בסיגנל startOfFrame כשעון איטי מבטיח שהתנועה לא תתבצע במקביל להצגה על המסך, דבר שהיה יוצר מריחה. לכן חישוב המסלול של האובייקט נעשה פעם אחת בכל סריקת מסך שלמה:



למשל, בתנועה ב- 45° , $Xspeed = Yspeed$, חישוב הקואורדינטה הבאה:

כל שעון איטי

$$X0 \leq X0 + Xspeed$$
$$Y0 \leq Y0 + Yspeed$$

כדי לשפר רזולוציה ניתן לבצע את חישובי המשוואות עם מספרים גדולים, על ידי הכפלתם בקבוע, ולהחזירם לפרופורציה המקורית על ידי חלוקה באותו קבוע לפני שמוציאים את התוצאה. הקפידו שהקבוע יהיה כפולה של 2, כגון 64, 256, וכו'.

אתחול
 $X0 \leq Xinit * 64$, $Y0 \leq Yinit * 64$
 כל שיעון איטי
 $X0 \leq X0 + Xspeed$, $Y0 \leq Y0 + Yspeed$
 בסוף לבצע עדכון אסינכרוני
 $X0out = X0 / 64$, $Y0out = Y0 / 64$

3.3. שכפול אובייקטים - פענוח XY

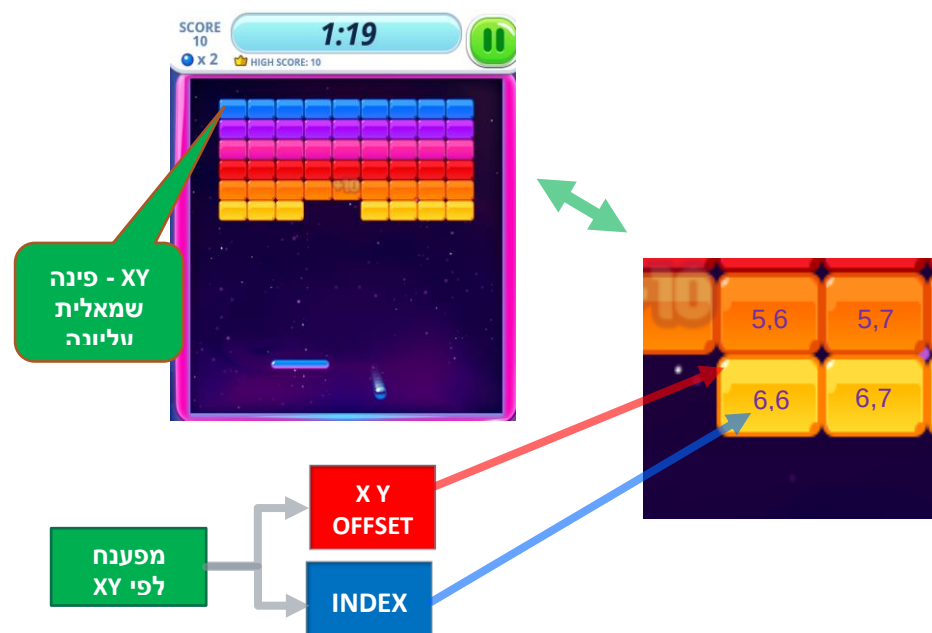
ישנה אפשרות לטיפול פשוט ויעיל במקרה שהרקע או האובייקטים אותם רוצים להציג על המסך מורכבים מכמה אלמנטים קטנים יותר אבל זהים בצורתם.

לשיכפול אובייקט: ניתן לייצר אובייקט בסיסי ולהציג אותו במיקומים הרצויים ואולי בצבעים שונים. הכתובת של האובייקט (בדוגמה להלן לפי הפינה השמאלית העליונה) צריכה לציין (ראה איור להלן):

- מיקום האובייקט - (INDEX) של הכתובת - DIV

- מיקום יחסי בתוך האובייקט - (OFFSET) של הכתובת - MOD

הערה: שימוש בפעולות DIV ו-MOD דורש משאבי מחשב רבים, על כן מומלץ לעבוד עד כמה שניתן עם חזקות של 2.

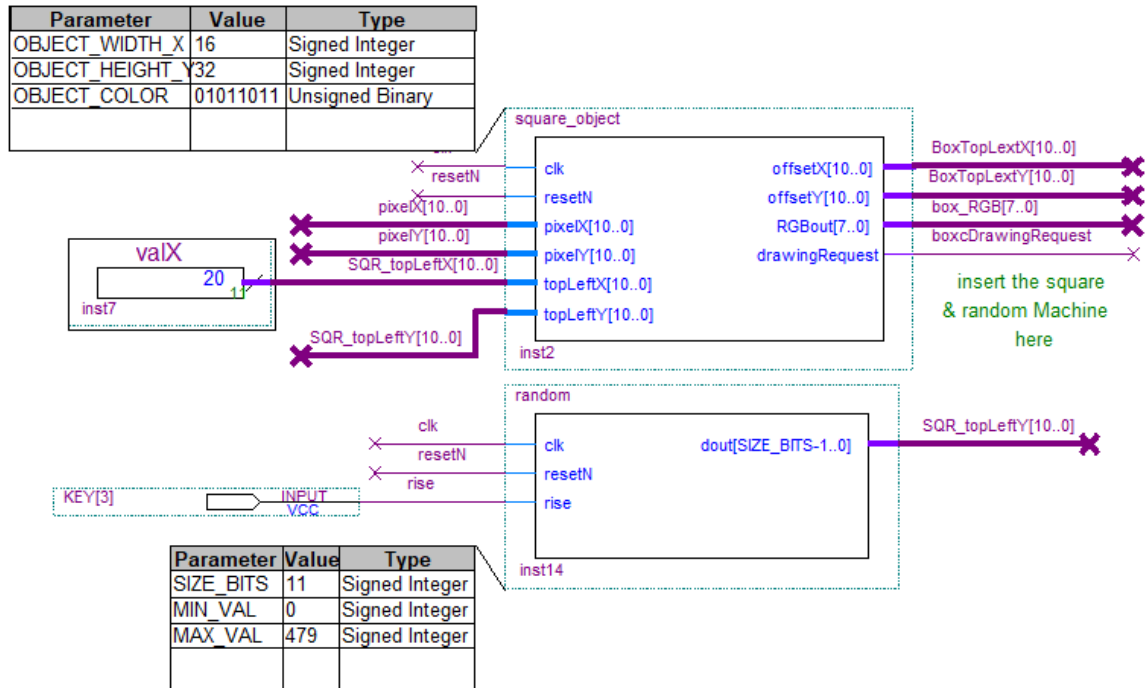


לפיענוח מיקום האובייקט: בצורה דומה אך הפוכה ניתן לפענח את מיקום האובייקט לפי הכתובת של הפינה השמאלית העליונה.

ז"א אם נתונה כתובת האובייקט, ניתן לחשב ממנה את שני הפרמטרים כמו קודם: את מיקום האובייקט לפי INDEX של הכתובת - DIV, ואת המיקום היחסי בתוך האובייקט לפי OFFSET של הכתובת - MOD.

3.1.1. אובייקטים נוספים מעל הרקע

ניתן להוסיף אובייקטים מעל הרקע. ביישום הנתון נוסף אובייקט בצורת מלבן, על ידי שמוש באותו מודול משרטוט הפרצופון.

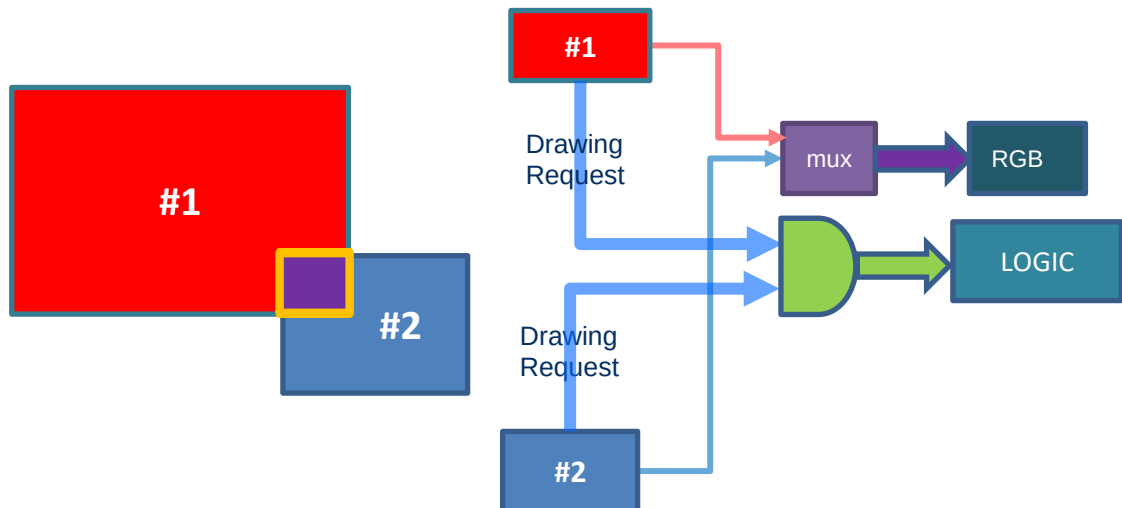


3.1.1.1 קביעת מיקום אקראי לאובייקט

ניתן לקבוע את המיקום שבו יוצב המלבן באופן אקראי, על ידי שימוש במכונת RANDOM אותה הכרת במעבדת DEBUG, כפי שנראה באיור הנ"ל. הפקת המספר האקראי נשלטת על ידי לחצן מהכרטיס.

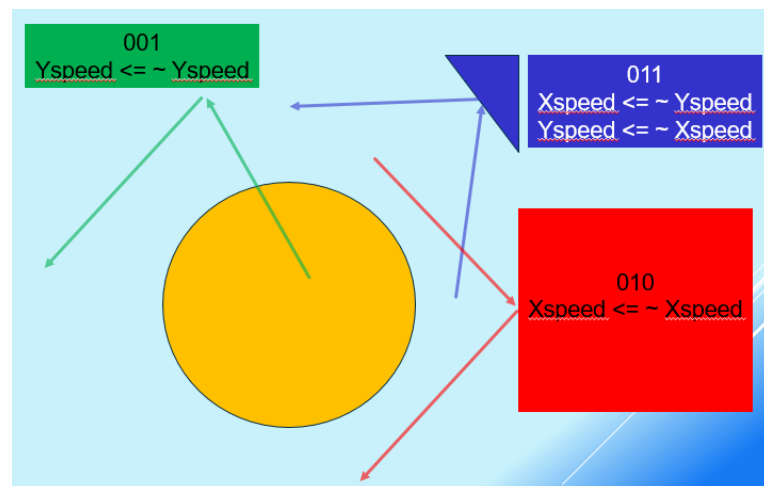
3.2. זיהוי התנגשות

כדי לזהות התנגשות בין שני אובייקטים ניתן להשתמש בבקשות התצוגה שלהם. כלומר אם שניים או יותר אובייקטים מבקשים להציג באותו זמן לאותו מקום אזי מודיעים על התנגשות, והמערכת תמשיך לטפל בכך לפי הצורך.



3.2.1. דוגמה של התנגשות כדור במסגרת

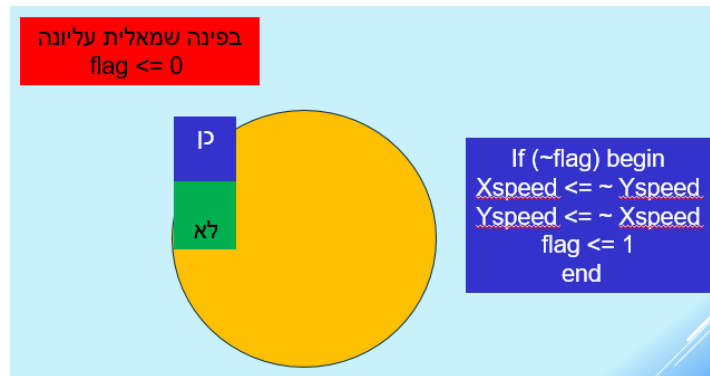
בדוגמה של התנגשות כדור במסגרת המגרש, שיטה לעידכון כיוון התנועה של הכדור אחרי ההתנגשות היא קידוד ב"צבע" של המסגרת, מה שמחייב כיוון החזרה מסויים, כפי שמודגם בדיאגרמה הבאה:



עקרונות הקידוד קובעים שלכל דופן ופינה יש צבע אחר והתנועה תהיה תמיד ב- 45° ללא קשר באיזה מקום בכדור פגע.

3.2.2. טיפול בהתנגשות כפולה

בעייה - אם בזמן בדיקת התנגשויות ישנו מגע בו זמנית בשני פיקסלים, מתקבלות שתי הפיכות והעצם ממשיך ישר מה שגורם לטיפול לא נכון בהתנגשות פתרון - להוסיף דגל שמאפרש רק שינוי אחד בכל פעם, כמו בדיאגרמה הבאה:



3.3. שכפול אובייקטים

תחילה, חשוב להבין את ההבדל בין SV לבין למשל שפת C בהקשר של שכפול - בקוד כמו בשפת C כאשר רוצים לשכפל משהו מסויים, נהוג לשים אותו בתוך לולאה או ליצור פונקציה שבה יהיה מימוש הקוד וזה כביכול הופך את הקוד לקומפקטי יותר (לא בהכרח מהיר יותר).

כאשר אנחנו מדברים על SV, שפה שמתארת חומרה, צריך להבין שהמשמעות של שכפול היא שכפול של לוגיקה ולכן צריך לחשוב האם יש צורך לשכפל לוגיקה או לא. קומפקטיות של הקוד פה היא שיקול פחות חשוב שכן למשל לולאת FOR יכולה להיות מאוד יעילה מבחינת כתיבה, אך מבחינת חומרה היא מייצרת שכפול גדול מאוד של חומרה שהוא לא בהכרח נדרש.

כשאנחנו מדברים על אובייקט שאנו רוצים לשכפל, כמו עץ או מדרגה, צריך לזכור שעבור כל אובייקט כזה אנחנו צריכים להחזיק מידע לגבי המיקום שלו (לפחות נקודה אחת במסך) ולכן זה דורש חומרה שלא ניתן להימנע ממנה. כדי לתאר ולממש אובייקטים שחוזרים על עצמם מספר רב של פעמים יש 2 גישות אפשריות, כאשר לכל אחת היתרונות והחסרונות שלה:

1. האפשרות הראשונה היא מימוש של אובייקט אחד בודד שמתפקד כמו שאנחנו רוצים (כאשר מקפידים שיהיה גנרי ככל שאפשר - כלומר המיקום ההתחלתי שלו, המהירות שלו וכו' ייקבעו על ידי פרמטרים חיצוניים או כניסות ולא בתוך המודול עצמו כקבועים). לאחר מימוש אובייקט יחיד, ניתן ליצור ממנו סימבול ולשכפל את המופעים שלו כמספר הפעמים שרוצים שהאובייקט יוכל להופיע במשחק. זה כמובן שכפול של לוגיקה אבל זהו שכפול הכרחי - אם אני רוצה לשכפל מספר עצים שנעים כל אחד במהירות שונה ונמצאים במיקומים שונים, אני חייב עבור כל עץ יחידה שתשמור את המיקום הנוכחי שלו וכן תעדכן את המיקום הבא שלו בהתאם לכניסות והפרמטרים הספציפיות שלו. לכן, למרות שזה נראה שכפול של מספר בלוקים זהים - זה לא אומר שזה לא יעיל. כאשר עושים שכפול כזה, הטעות הנפוצה היא לשכפל גם את אובייקט ה BM כמספר היחידות, למרות שאובייקט זה מכיל מטריצה גדולה שזהה לכל היחידות. במקום זה, מה שצריך לעשות זה ליצור יחידת BM בודדת אליה נכנסים כל יציאות ה square object של כל האובייקטים המשוכפלים ויחידת ה BM משמשת גם כ MUX שבוחר מתוך כולם איזו בקשה לצייר, מה הצבע וה DR שצריך להוציא. (בדרך כלל כשמשכפלים אובייקטים הם לא נמצאים אחד על השני ולכן בכל מחזור שעון אנחנו נטפל לכל היותר בבקשה אחת, שכן בכל מחזור מטפלים בפיקסל אחד).

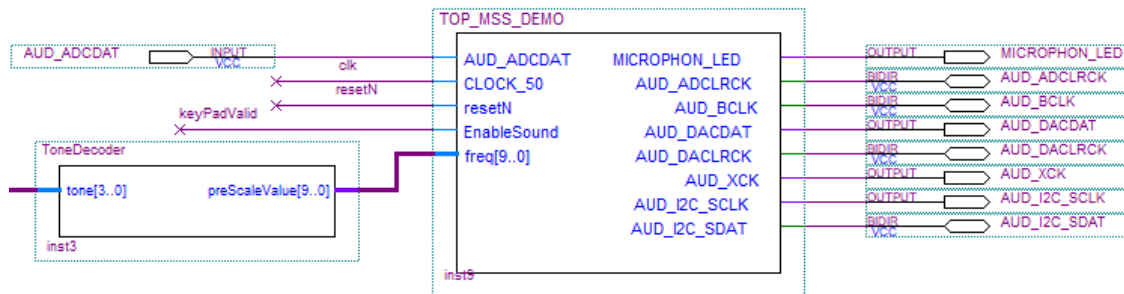
גם אם יש 2 אובייקטים זהים שנמצאים אחד על השני, בסוף יחידת ה BM תצטרך להחליט איזה מהם מציגים ואיזה מהם לא מציגים (מה נמצא מקדימה ומה מאחורה) ולכן בכל מקרה טבלת BM בודדת תספיק.

2. האפשרות השנייה היא חלוקה של המסך למספר קבוע של איזורים כך שבכל איזור נוכל להחליט האם יש אובייקט או אין אובייקט. למשל, חלוקת המסך לריבועים של 32×32 פיקסלים תגרום לחלוקת המסך ל 20×15 איזורים (וחלוקה של 16×16 פיקסלים תגרום לחלוקה של המסך ל 40×30 איזורים). ניתן להחזיק מטריצה שבכל אחד מהאיזורים האלה תחזיק ערך שיסמן האם יש או אין שם אובייקט (למשל יש או אין מדרגה). ניתן גם להחזיק מטריצה קטנה יותר שתייצג רק איזור מסויים במסך, כך למשל אפשר להחזיק מטריצה של 10×10 שכל תא ייצג 16×16 פיקסלים ולכן בסך הכל המטריצה תייצג איזור של 160×160 פיקסלים שיתחילו מנקודה מסויימת על המסך (למשל ממיקום (40,40) עד למיקום (200,200)). היתרון של השיטה הזאת הוא שאנו יכולים לייצר הרבה מאוד אובייקטים בלי לשמור את המיקום המדויק של הפינה השמאלית העליונה של כל אחד (אלא רק המיקום שלהם בטבלה המוקטנת, של 20×15 למשל), אך מצד שני זה מוריד לנו את הגמישות מבחינת המיקומים האפשריים שאובייקט יכול להופיע בהם - שכן יש רק 20×15 (או 40×30) מיקומים אפשריים. כאשר אנחנו מקבלים מה VGA פיקסל אותו אנו רוצים לצייר, אנו ממפים אותו (על ידי פעולת חילוק) לתא המתאים במטריצה ואז יודעים האם יש או אין אובייקט באותו מקום. ניתן בצורה זאת גם להחליט שלא דווקא כל תא בודד ייצג אובייקט, אלא יתכן גם אובייקט שיהיה מורכב מכמה תאים סמוכים למשל, או מכמה סוגים של אובייקטים שונים (לשם כך צריך לתת מספר שונה במטריצה לכל סוג של תא). לאחר מכן על סמך מידע זה מעבירים/לא מעבירים ליחידת ה BM בקשה לציור (את האופסט מקבלים מפעולת המודולו). שימו לב שכיוון שאנו משתמשים בפעולות מודולו וחילוק, שימוש במספרים שאינם חזקה של 2 יהיו מאוד לא יעילים מבחינת הלוגיקה שצריך על מנת לממש אותם ואילו בעבודה עם חזקות של 2 פעולות אלו הן בסך הכל הזזות של ביטים.

לסיכום, ניתן להשתמש בכל אחת מהגישות הנ"ל בשביל כל אחד מהשימושים. אפשר לומר שבמקרה של אובייקטים שנעים באופן רציף, שיכולים להיות ממוקמים בכל מקום ואין הרבה כאלו במסך בו זמנית (כמה בודדים) הגישה הראשונה עדיפה. לעומת זאת במקרה של אובייקטים קבועות במקום, אשר המיקומים שלהם קבועים ויש הרבה כאלו במסך בו זמנית, הגישה של מטריצה דווקא עדיפה.

3.4. הוספת צלילים

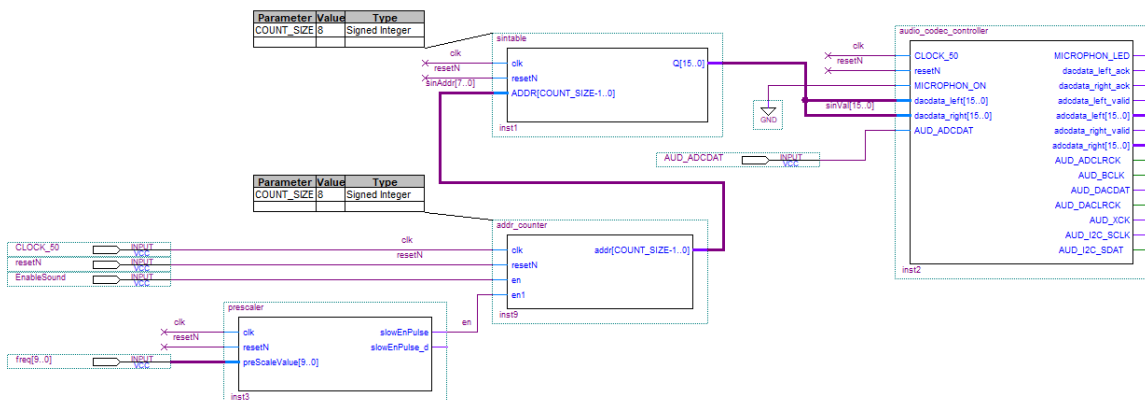
יישום הפרצופון כולל גם ממשק שמע שנתון לך בכללותו, TOP_MSS_DEMO.



יישום זה, TOP_MSS_DEMO, משתמש בפלטפורמת ה- MSS (Mixed Signals System) של הכרטיס ומייצר את שמע.

3.4.1. מחולל הצליל - גל סינוס

יישום זה מייצר את סינוס דיגיטלי אותו המערכת ממירה לאות אנלוגי. התדר של אות הסינוס קובע את הצליל הנשמע והמשתמש יכול לקבוע אותו. מחולל הצליל נתון להלן פועל באופן הבא:



- הרכיב audio_codec_controller הוא בקר השמע שממיר את האות הדיגיטלי לאנלוגי. אין לשנות אותו או את הכניסות/יציאות שלו.
- הרכיב sin_table הוא זיכרון בגודל 256X16 שמכיל 256 דגימות של מחזור סינוס אחד.
- הרכיב addr_counter הוא מונה שעובר מחזורית על הכתובות של טבלת הסינוס ואז הצליל מושמע, כל עוד כניסת ה- EnableSound היא ב- 1 לוגי.
- הרכיב prescaler הוא מחלק תדר של השעון המרכזי והוא קובע את התדר של אות הסינוס, כלומר את הצליל הנשמע.
- הכניסה freq היא וקטור של 10 ביט. הערך "n" שלו קובע את תדר הצליל המושמע.

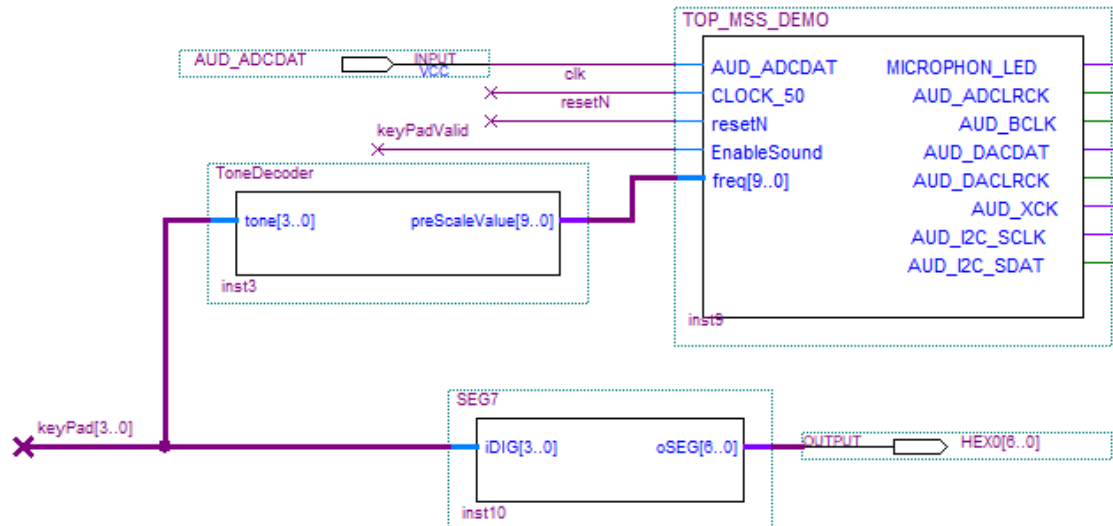
כדי לקבל את סינוס בתדר fHz מסויים, גורם החלוקה n של השעון יחושב לפי:

$$n = \frac{50,000,000}{256 \times f}$$

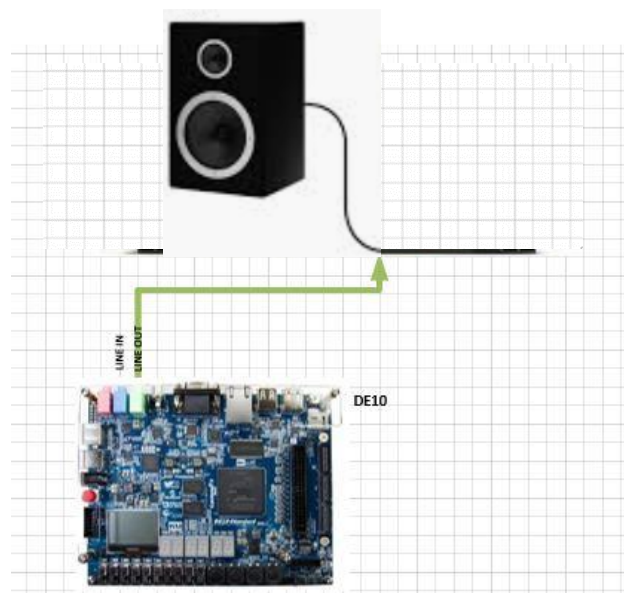
יש לציין שעם שעון מרכזי של 50 MHz התדר הגבוה ביותר שניתן להשיג הוא:

$$f_{\max} = \frac{50,000,000}{256}$$

במערכת הנתונה לך גורם חלוקה זה (n) הינו הכניסה **freq** של היחידה prescaler. היישום כולל גם מערכת ליצירת הצלילים של אוקטבה בסיסית.

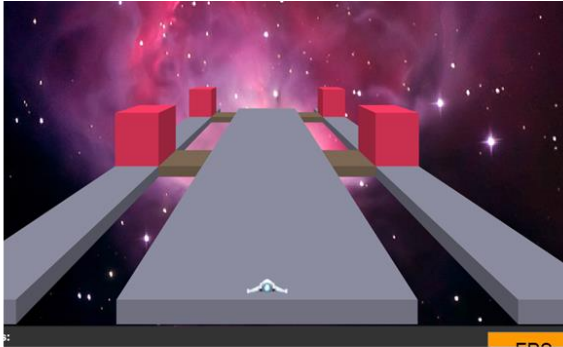


יציאת השמע של המערכת מתחברת להתקן שמע, כגון רמקול או אוזניות. באופן ספציפי, במעבדה, נשתמש ברמקולים חיצוניים או ברמקולים של המסך.



3.5. תלת מימד – איזומטריה

כדי לייצר תחושה של תלת מימד בתמונה יש לשרטט רקעים/אובייקטים בצורת טרפז. בדרך כלל הטרפז יהיה רחב בבסיסו, בתחתית המסך, ויהיה צר יותר בחלקו העליון של המסך. לדוגמה:



לציור הטרפז

1. עבור כל Y_i חישוב X_{iL} X_{iR}

2. בדיקה אם בתוך התחום

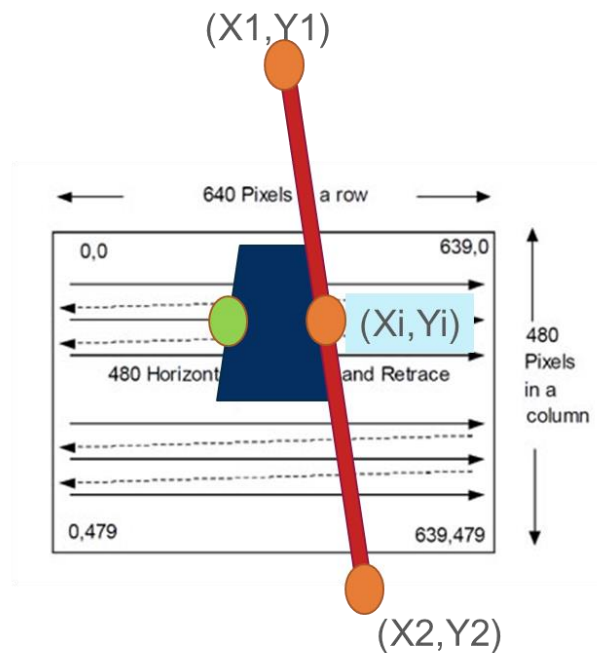
$$\text{Green Circle } (X_{iL} < X) \ \&\& \ (X < X_{iR}) \ \text{Orange Circle}$$

||

$$(top < Y) \ \&\& \ (Y < bottom)$$

שימו לב ש- $\frac{X_2 - X_1}{Y_2 - Y_1}$ כפולה של 2ⁿ

למשל 1/8



משוואת הישר:

$$X_i = X_1 + (Y_i - Y_1) \frac{X_2 - X_1}{Y_2 - Y_1}$$

4. נספח - טבלת המרה של צבעים בסיסיים

Hex to RGB color table

Color	Color name	Hex	(R,G,B)	(RGB-8bit)
	Black	#000000	(0,0,0)	(000_000_00)
	White	#FFFFFF	(255,255,255)	(111_111_11)
	Red	#FF0000	(255,0,0)	(111_000_00)
	Lime	#00FF00	(0,255,0)	(000_111_00)
	Blue	#0000FF	(0,0,255)	(000_000_11)
	Yellow	#FFFF00	(255,255,0)	(111_111_00)
	Cyan	#00FFFF	(0,255,255)	(000_111_11)
	Magenta	#FF00FF	(255,0,255)	(111_000_11)
	Silver	#C0C0C0	(192,192,192)	(101_101_10)
	Gray	#808080	(128,128,128)	(011_011_01)
	Maroon	#800000	(128,0,0)	(011_000_00)
	Olive	#808000	(128,128,0)	(011_011_00)
	Green	#008000	(0,128,0)	(000_011_00)
	Purple	#800080	(128,0,128)	(011_000_01)
	Teal	#008080	(0,128,128)	(000_011_01)
	Navy	#000080	(0,0,128)	(000_000_01)

5. נספח - טבלת המרה של כל צבעים

0 0		2 0		4 0		6 0		8 0		A 0		C 0		E 0
0 1		2 1		4 1		6 1		8 1		A 1		C 1		E 1
0 2		2 2		4 2		6 2		8 2		A 2		C 2		E 2
0 3		2 3		4 3		6 3		8 3		A 3		C 3		E 3
0 4		2 4		4 4		6 4		8 4		A 4		C 4		E 4
0 5		2 5		4 5		6 5		8 5		A 5		C 5		E 5
0 6		2 6		4 6		6 6		8 6		A 6		C 6		E 6
0 7		2 7		4 7		6 7		8 7		A 7		C 7		E 7
0 8		2 8		4 8		6 8		8 8		A 8		C 8		E 8
0 9		2 9		4 9		6 9		8 9		A 9		C 9		E 9
0 A		2 A		4 A		6 A		8 A		A A		C A		E A
0 B		2 B		4 B		6 B		8 B		A B		C B		E B
0 C		2 C		4 C		6 C		8 C		A C		C C		E C
0 D		2 D		4 D		6 D		8 D		A D		C D		E D
0 E		2 E		4 E		6 E		8 E		A E		C E		E E
0 F		2 F		4 F		6 F		8 F		A F		C F		E F
1 0		3 0		5 0		7 0		9 0		B 0		D 0		F 0
1 1		3 1		5 1		7 1		9 1		B 1		D 1		F 1
1 2		3 2		5 2		7 2		9 2		B 2		D 2		F 2
1 3		3 3		5 3		7 3		9 3		B 3		D 3		F 3
1 4		3 4		5 4		7 4		9 4		B 4		D 4		F 4
1 5		3 5		5 5		7 5		9 5		B 5		D 5		F 5
1 6		3 6		5 6		7 6		9 6		B 6		D 6		F 6
1 7		3 7		5 7		7 7		9 7		B 7		D 7		F 7
1 8		3 8		5 8		7 8		9 8		B 8		D 8		F 8
1 9		3 9		5 9		7 9		9 9		B 9		D 9		F 9
1 A		3 A		5 A		7 A		9 A		B A		D A		F A
1 B		3 B		5 B		7 B		9 B		B B		D B		F B
1 C		3 C		5 C		7 C		9 C		B C		D C		F C
1 D		3 D		5 D		7 D		9 D		B D		D D		F D
1 E		3 E		5 E		7 E		9 E		B E		D E		F E
1 F		3 F		5 F		7 F		9 F		B F		D F		F F